

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	A. Mohith krishna	Reg. No.:	192425199
Section / Batch:	2024	Date:	12-08-2026

Code of Conduct

I A. Mohith krishna (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Mohith krishna

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	4	4	3	3	3	17	85
2	4	3	4	3	3	17	
3	3	4	3	3	4	17	
4	3	4	4	3	3	17	
5	3	4	3	3	4	17	

Faculty In-charge		Program Director
Dr. Kumaragurubaran. T		
Signature: _____	PROGRAM:	Signature: _____
Date: _____	corpus = [	Date: _____

PROGRAM:

```
corpus = [
    ("The boy eats rice".split(), ["DT","NN","VBZ","NN"]),
    ("The girl drinks milk".split(), ["DT","NN","VBZ","NN"]),
    ("A cat drinks milk".split(), ["DT","NN","VBZ","NN"]),
    ("The dog chases cat".split(), ["DT","NN","VBZ","NN"]),
    ("A teacher teaches students".split(), ["DT","NN","VBZ","NNS"]),
    ("Students study English".split(), ["NNS","VBP","NN"]),
    ("Birds fly high".split(), ["NNS","VBP","RB"]),
    ("Children play games".split(), ["NNS","VBP","NNS"])
]
```

```
emission = {}
transition = {}
tag_count = {}
start_count = {}
```

for words, tags in corpus:

```
    first_tag = tags[0]
    start_count[first_tag] = start_count.get(first_tag, 0) + 1
```

```
    prev = "START"
```

```
    for w, t in zip(words, tags):
```

```
        tag_count[t] = tag_count.get(t, 0) + 1
```

```
        emission.setdefault(t, {})
        emission[t][w] = emission[t].get(w, 0) + 1
```

```
transition.setdefault(prev, {})  
transition[prev][t] = transition[prev].get(t, 0) + 1
```

```
prev = t
```

```
print("\nEmission Probability\n")
```

```
for tag in emission:
```

```
    print(tag)
```

```
    for word in emission[tag]:
```

```
        p = emission[tag][word] / tag_count[tag]
```

```
        print(word, "=", round(p, 3))
```

```
    print()
```

```
print("\nTransition Probability\n")
```

```
for t1 in transition:
```

```
    total = sum(transition[t1].values())
```

```
    print(t1)
```

```
    for t2 in transition[t1]:
```

```
        p = transition[t1][t2] / total
```

```
        print(t2, "=", round(p, 3))
```

```
    print()
```

```
sentence = "The cat drinks milk".split()
```

```
tags = list(emission.keys())
```

```
viterbi = []
```

```
backpointer = []
```

```
first_word = sentence[0]
```

```
first_values = {}
```

```
first_paths = {}
```

```
for tag in tags:
```

```
    if tag in start_count:
```

```
        start_prob = start_count[tag] / len(corpus)
```

```
    else:
```

```

start_prob = 0

if first_word in emission[tag]:
    emission_prob = emission[tag][first_word] / tag_count[tag]
else:
    emission_prob = 0

first_values[tag] = start_prob * emission_prob
first_paths[tag] = ""

viterbi.append(first_values)
backpointer.append(first_paths)

for i in range(1, len(sentence)):

    word = sentence[i]

    current_values = {}
    current_paths = {}

    for current_tag in tags:

        if word in emission[current_tag]:
            emission_prob = emission[current_tag][word] / tag_count[current_tag]
        else:
            emission_prob = 0

        best_prob = 0
        best_tag = ""

        for previous_tag in tags:

            if current_tag in transition.get(previous_tag, {}):

                total = sum(transition[previous_tag].values())

                transition_prob = (
                    transition[previous_tag][current_tag] / total
                )

            else:
                transition_prob = 0

            probability = (
                viterbi[i - 1][previous_tag]
                * transition_prob
                * emission_prob
            )

            if probability > best_prob:

                best_prob = probability
                best_tag = previous_tag

        current_values[current_tag] = best_prob

```

```
current_paths[current_tag] = best_tag
```

```
viterbi.append(current_values)  
backpointer.append(current_paths)
```

```
best_tag = ""  
best_prob = 0
```

```
for tag in tags:
```

```
    if viterbi[-1][tag] > best_prob:
```

```
        best_prob = viterbi[-1][tag]  
        best_tag = tag
```

```
result = [best_tag]
```

```
for i in range(len(sentence) - 1, 0, -1):
```

```
    best_tag = backpointer[i][best_tag]
```

```
    result.insert(0, best_tag)
```

```
print("\nSentence :", sentence)  
print("Predicted Tags :", result)
```

OUTPUT:

```
Words: ['Birds', 'fly', 'high']
Tags : ['NNS', 'VBP', 'RB']

...

Words: ['Children', 'play', 'games']
Tags : ['NNS', 'VBP', 'NNS']
```

EMISSION PROBABILITIES

```
-----
P(The | DT) = 0.6
P(boy | NN) = 0.1
P(eats | VBZ) = 0.2
P(rice | NN) = 0.1
P(girl | NN) = 0.1
P(drinks | VBZ) = 0.4
P(milk | NN) = 0.2
P(A | DT) = 0.4
P(cat | NN) = 0.2
P(dog | NN) = 0.1
P(chases | VBZ) = 0.2
P(teacher | NN) = 0.1
P(teaches | VBZ) = 0.2
P(students | NNS) = 0.2
P(Students | NNS) = 0.2
P(study | VBP) = 0.3333333333333333
P(English | NN) = 0.1
P(Birds | NNS) = 0.2
P(fly | VBP) = 0.3333333333333333
P(high | RB) = 1.0
P(Children | NNS) = 0.2
P(play | VBP) = 0.3333333333333333
P(games | NNS) = 0.2
```

TRANSITION PROBABILITIES

```
-----
P(NN | DT) = 1.0
P(VBZ | NN) = 1.0
P(NN | VBZ) = 0.8
P(VBZ | VBZ) = 0.2
```